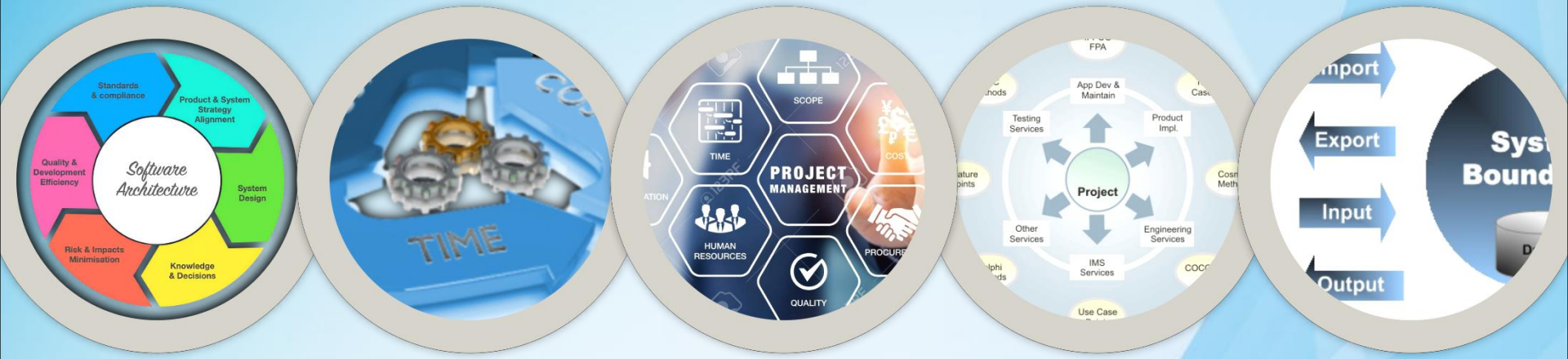


Software Architecture Introduction

Architecture Styles

By: Võ Văn Hải

Email: vovanhai@ueh.edu.vn

Introduction

- Architecture styles, sometimes called architecture patterns, describe a named relationship of components covering a variety of architecture characteristics.
- An architecture style name, similar to design patterns, creates a single name that acts as shorthand between experienced architects.
  - For example, when an architect talks about a layered monolith, their target in the conversation understands aspects of structure, which kinds of architecture characteristics work well (and which ones can cause problems), typical deployment models, data strategies, and a host of other information. Thus, architects should be familiar with the basic names of fundamental generic architecture styles.
- Each name captures a wealth of understood detail, one of the purposes of design patterns. An architecture style describes the topology, assumed and default architecture characteristics, both beneficial and detrimental.
- Many common modern architecture patterns was covered in this section. However, architects should be familiar with several fundamental patterns that appear embedded within the larger patterns.

Chapter 9
Foundations

Fundamental Patterns

Big Ball of Mud

- Architects refer to the absence of any discernible architectural structure as a Big Ball of Mud, named after the eponymous anti-pattern defined in a paper released in 1997 by Brian Foote and Joseph Yoder.
- A big ball of mud might describe a simple scripting application with event handlers wired directly to database calls, with no internal structure. Many trivial applications start like this and become unwieldy as they grow.
- Architects generally want to avoid this type of architecture at all costs. The lack of structure makes change increasingly tricky. This type of architecture also suffers from problems in deployment, testability, scalability, and performance.

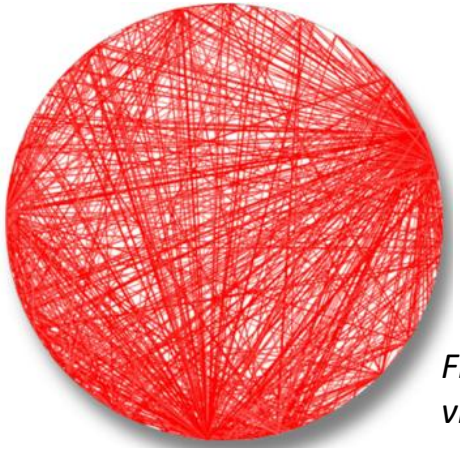


Figure 9-1. A Big Ball of Mud architecture visualized from a real code base

Fundamental Patterns

Unitary Architecture

- When software originated, there was only the computer, and software ran on it. Through the various eras of hardware and software evolution, the two started as a single entity, then split as the need for more sophisticated capabilities grew.
- Few unitary architectures exist outside embedded systems and other highly constrained environments.
- Generally, software systems tend to grow in functionality over time, requiring separation of concerns to maintain operational architecture characteristics, such as performance and scale.

Fundamental Patterns

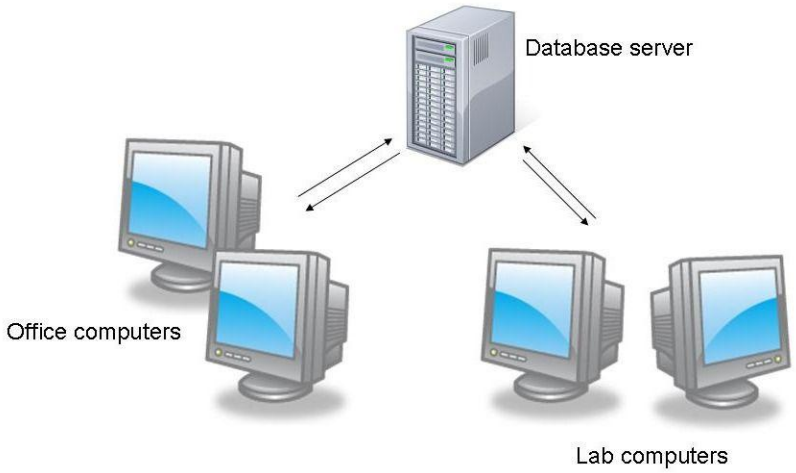
Client/Server

- A fundamental style in architecture separates technical functionality between front-end and back-end, called a two-tier, or client/server, architecture.
- Many different flavors of this architecture exist, depending on the era and computing capabilities:
  - Desktop + database server
  - Browser + web server
  - Three-tier

Fundamental Patterns

Client/Server - Desktop + database server

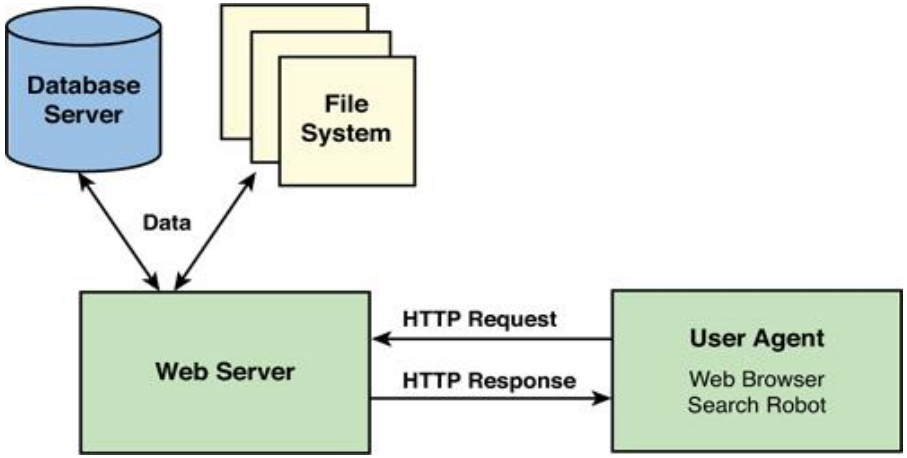
- An early personal computer architecture encouraged developers to write rich desktop applications in user interfaces like Windows, separating the data into a separate database server.
- This architecture coincided with the appearance of standalone database servers that could connect via standard network protocols. It allowed presentation logic to reside on the desktop, while the more computationally intense action (both in volume and complexity) occurred on more robust database servers.



Fundamental Patterns

*Client/Server - **Browser + web server***

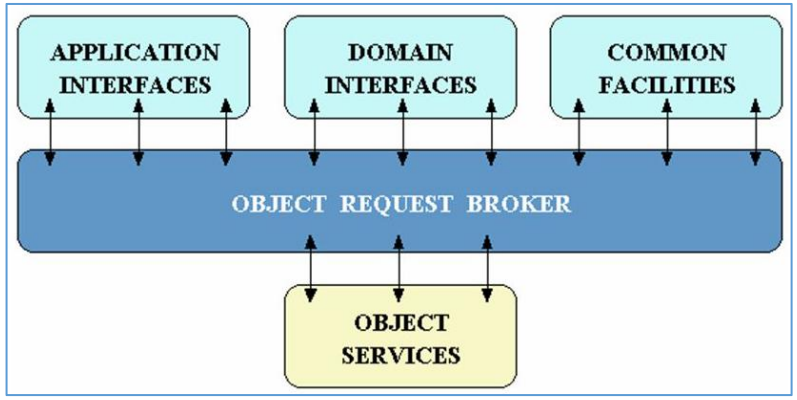
- Once modern web development arrived, the typical split became a web browser connected to the web server (which in turn was connected to a database server). The separation of responsibilities was similar to the desktop variant but with even thinner clients as browsers, allowing a wider distribution both inside and outside firewalls.
- Even though the database is separate from the web server, architects often still consider this a two-tier architecture because the web and database servers run on one class of machine within the operations center, and the user interface runs on the user's browser.



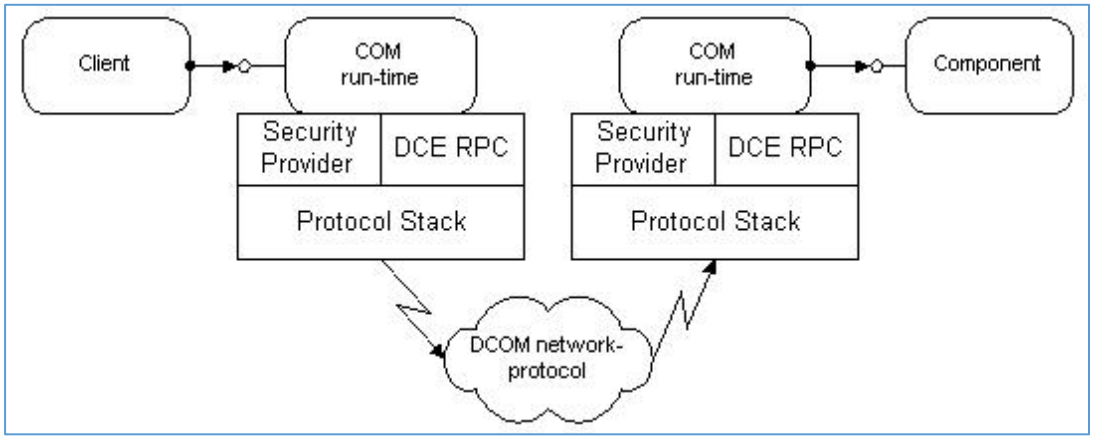
Fundamental Patterns

Client/Server - Three-tier

- An architecture that became quite popular during the late 1990s was a three-tier architecture, which provided even more layers of separation.
- As tools like application servers became popular in Java and .NET, companies started building even more layers in their topology:
  - a database tier using an industrial-strength database server,
  - an application tier managed by an application server,
  - frontend coded in generated HTML, and increasingly, JavaScript, as its capabilities expanded.



Common Object Request Broker Architecture (**CORBA**) model



Distributed Component Object Model (**DCOM**)

Monolithic Versus Distributed Architectures

Introduction

- Architecture styles can be classified into two main types:
  - monolithic (single deployment unit of all code) and
  - distributed (multiple deployment units connected through remote access protocols).
- While no classification scheme is perfect, distributed architectures all share a common set of challenges and issues not found in the monolithic architecture styles, making this classification scheme a good separation between the various architecture styles. In this book we will describe in detail the following architecture styles:
  - Monolithic
    - Layered architecture
    - Pipeline architecture
    - Microkernel architecture
  - Distributed
    - Service-based architecture
    - Event-driven architecture
    - Space-based architecture
    - Service-oriented architecture
    - Microservices architecture

Monolithic Versus Distributed Architectures

Fallacy #1: The Network Is Reliable

- Developers and architects alike assume that the network is reliable, but it is not. While networks have become more reliable over time, the fact of the matter is that networks still remain generally unreliable. This is significant for all distributed architectures because all distributed architecture styles rely on the network for communication to and from services and between services.
- The more a system relies on the network (such as microservices architecture), the potentially less reliable it becomes.

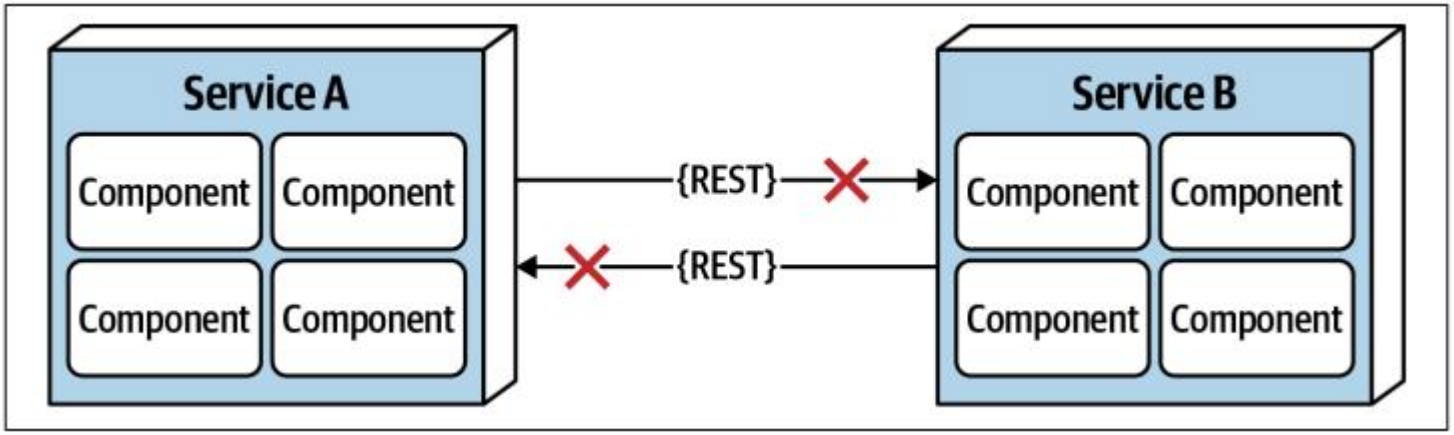


Figure 9-2. The network is not reliable

Monolithic Versus Distributed Architectures

Fallacy #2: Latency Is Zero

- When a local call is made to another component via a method or function call, that time (t_local) is measured in nanoseconds or microseconds. However, when that same call is made through a remote access protocol (such as REST, messaging, or RPC), the time measured to access that service (t_remote) is measured in milliseconds. Therefore, t_remote will always be greater that t_local.
- Latency in any distributed architecture is not zero, yet most architects ignore this fallacy, insisting that they have fast networks.

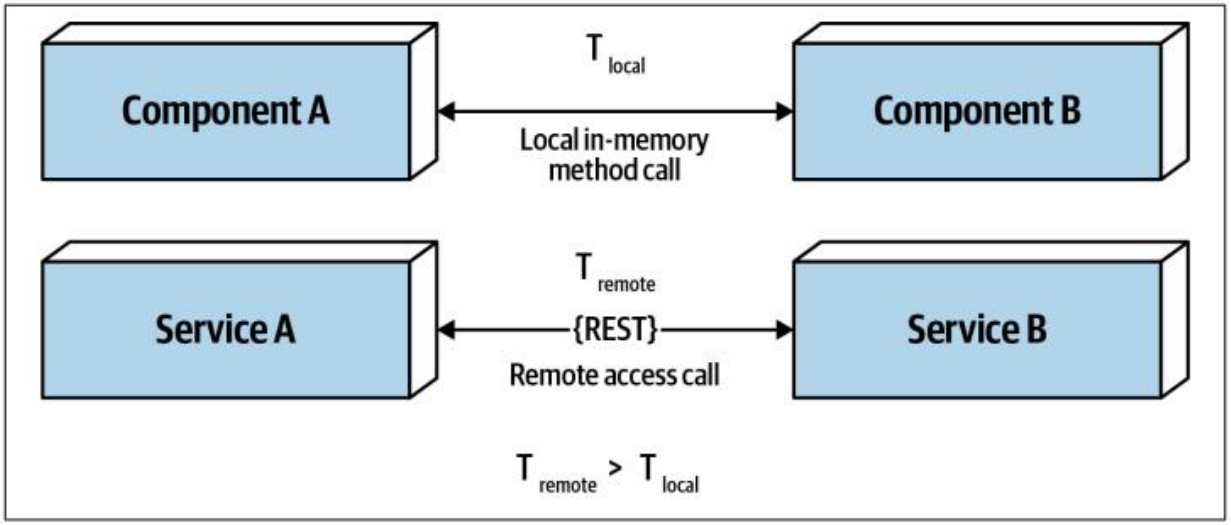


Figure 9-3. Latency is not zero

When using any distributed architecture, architects must know this latency average.

Monolithic Versus Distributed Architectures

Fallacy # 3: Bandwidth Is Infinite

- Bandwidth is usually not a concern in monolithic architectures, because once processing goes into a monolith, little or no bandwidth is required to process that business request.
- However, as shown in Figure 9-4, once systems are broken apart into smaller deployment units (services) in a distributed architecture such as microservices, communication to and between these services significantly utilizes bandwidth, causing networks to slow down, thus impacting latency (fallacy #2) and reliability (fallacy #1).

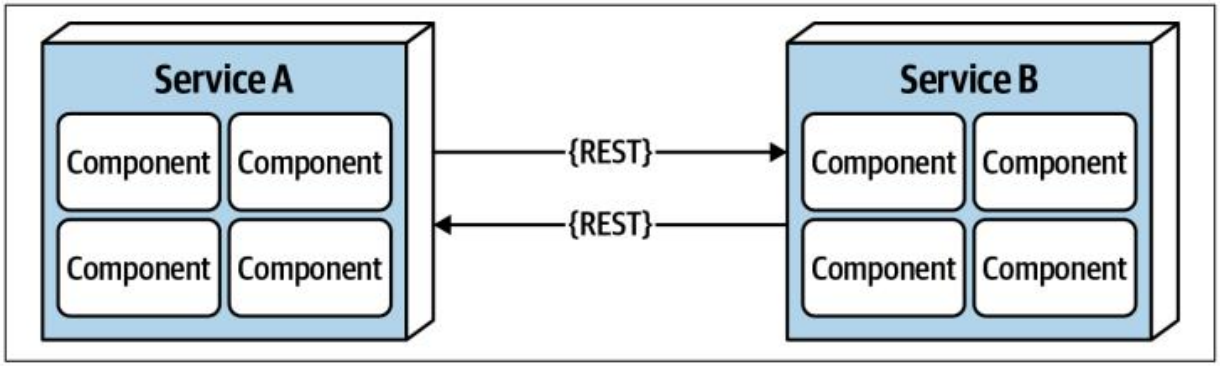


Figure 9-4. Bandwidth is not infinite

Monolithic Versus Distributed Architectures

Fallacy # 4: The Network Is Secure

- Most architects and developers get so comfortable using virtual private networks (VPNs), trusted networks, and firewalls that they tend to forget about this fallacy of distributed computing: the network is not secure. Security becomes much more challenging in a distributed architecture.
- The surface area for threats and attacks increases by magnitudes when moving from a monolithic to a distributed architecture. Having to secure every endpoint, even when doing interservice communication, is another reason performance tends to be slower in synchronous, highly-distributed architectures such as microservices or service-based architecture.

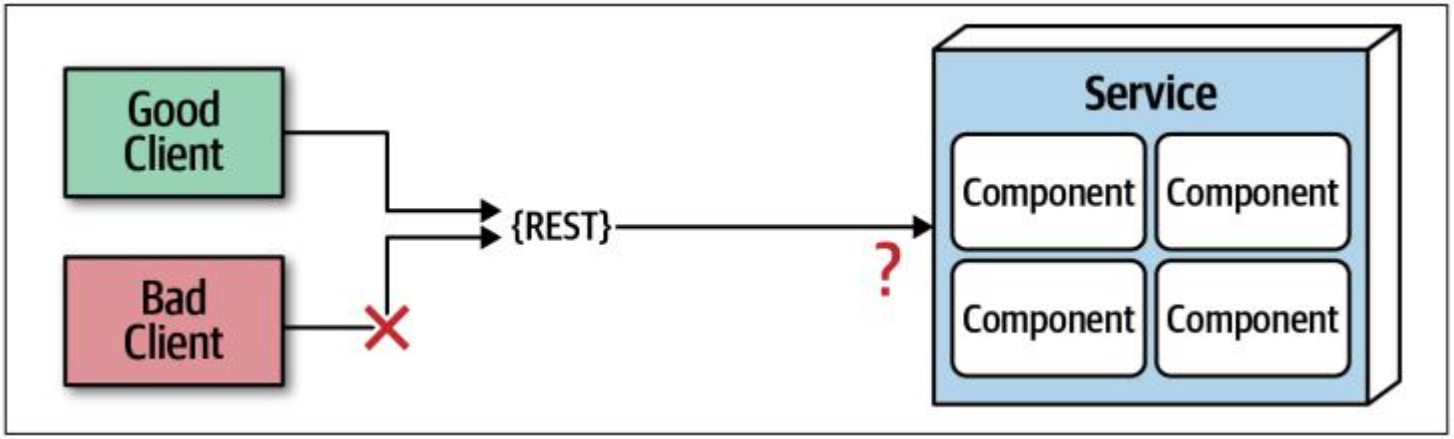


Figure 9-5. The network is not secure

Monolithic Versus Distributed Architectures

Fallacy # 5: The Topology Never Changes

- This fallacy refers to the overall network topology, including all of the routers, hubs, switches, firewalls, networks, and appliances used within the overall network. Architects assume that the topology is fixed and never changes. Of course it changes. It changes all the time.
- Architects must be in constant communication with operations and network administrators to know what is changing and when so that they can make adjustments accordingly to reduce the type of surprise previously described.

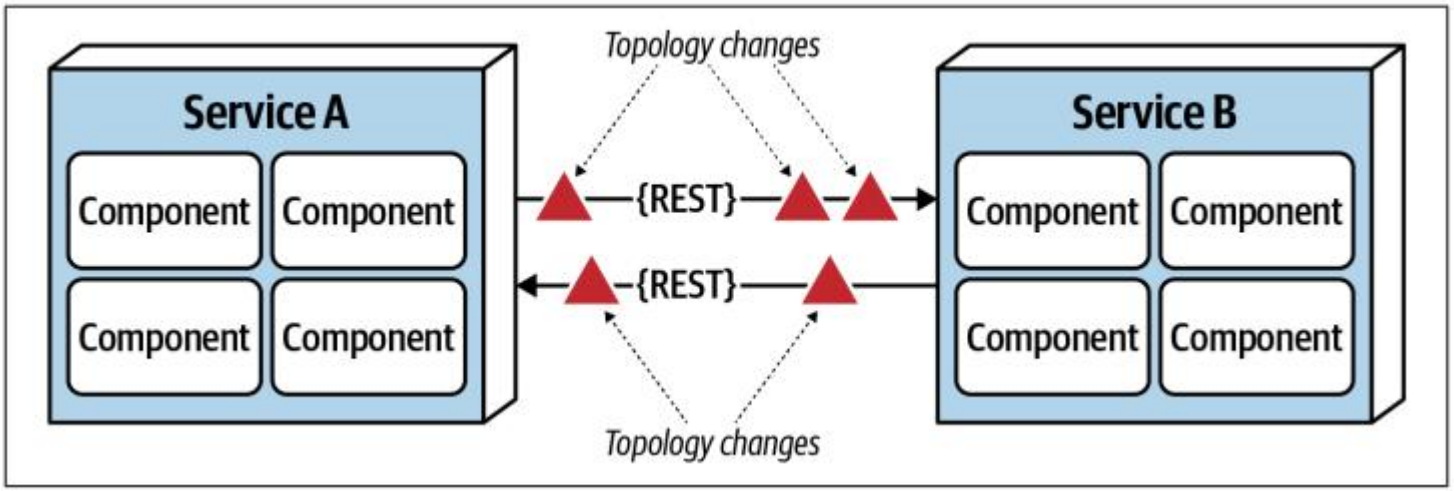


Figure 9-6. The network topology always changes

Monolithic Versus Distributed Architectures

Fallacy # 6: There Is Only One Administrator

- Architects all the time fall into this fallacy, assuming they only need to collaborate and communicate with one administrator. But: There are dozens of network administrators in a typical large company.
- This fallacy points to the complexity of distributed architecture and the amount of coordination that must happen to get everything working correctly.
- Monolithic applications do not require this level of communication and collaboration due to the single deployment unit characteristics of those architecture styles.

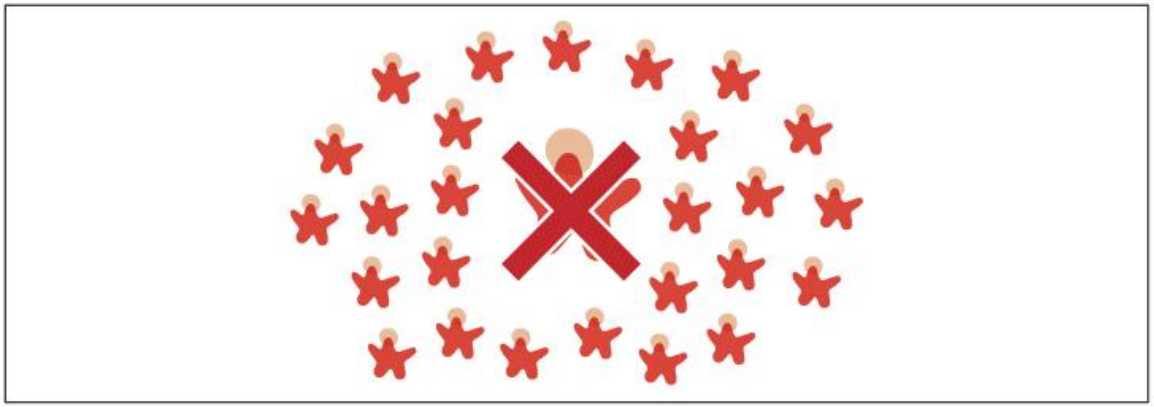


Figure 9-7. There are many network administrators, not just one

Monolithic Versus Distributed Architectures

Fallacy # 7: Transport Cost Is Zero

- Transport cost here does not refer to latency, but rather to actual cost in terms of money associated with making a "simple RESTful call."
- Architects assume (incorrectly) that the necessary infrastructure is in place and sufficient for making a simple RESTful call or breaking apart a monolithic application. It is usually not.
- Whenever embarking on a distributed architecture, we encourage architects to analyze the current server and network topology with regard to capacity, bandwidth, latency, and security zones to not get caught up in the trap of surprise with this fallacy.

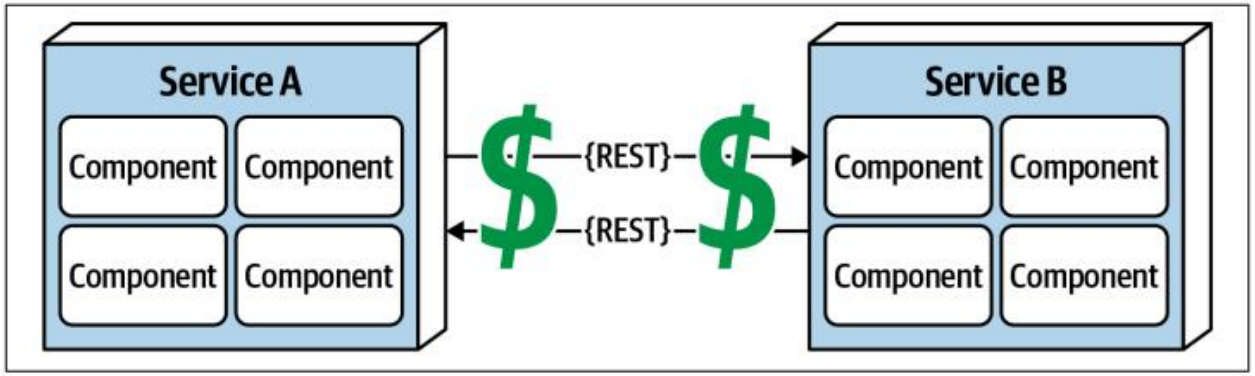


Figure 9-8. Remote access costs money

Monolithic Versus Distributed Architectures

Fallacy # 8: The Network Is Homogeneous

- There are may hardware vendors. The significance of this fallacy is that not all of those heterogeneous hardware vendors play together well.
  - Most of it works, but does Juniper hardware seamlessly integrate with Cisco hardware?
- Networking standards have evolved over the years, making this less of an issue, but the fact remains that not all situations, load, and circumstances have been fully tested, and as such, network packets occasionally get lost.

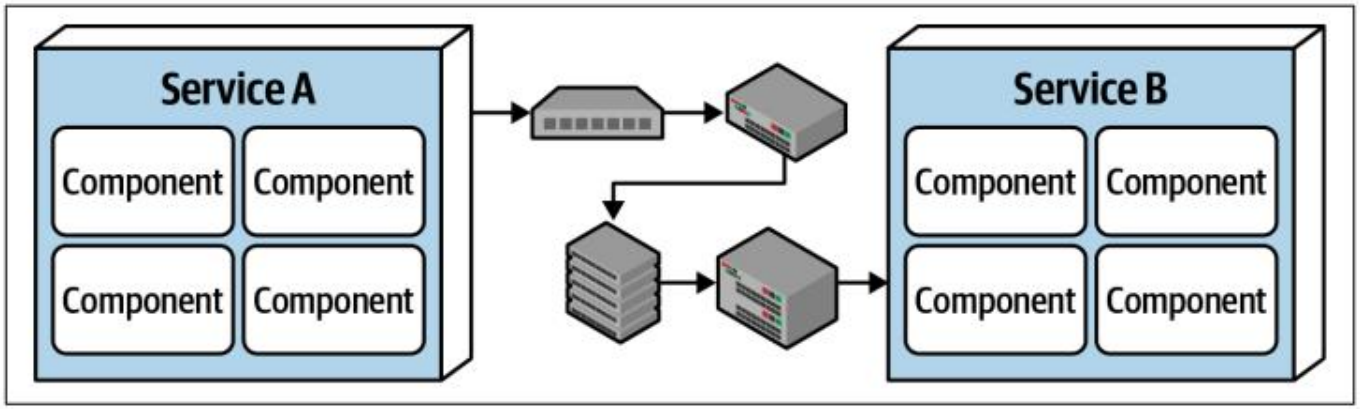
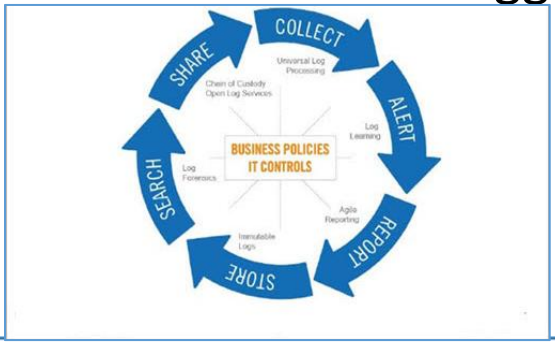


Figure 9-9. The network is not homogeneous

Monolithic Versus Distributed Architectures

Other Distributed Considerations - Distributed logging

- Performing root-cause analysis to determine why a particular order was dropped is very difficult and time-consuming in a distributed architecture due to the distribution of application and system logs.
  - In a monolithic application there is typically only one log, making it easier to trace a request and determine the issue.
  - However, distributed architectures contain dozens to hundreds of different logs, all located in a different place and all with a different format, making it difficult to track down a problem.
- Logging consolidation tools such as *Datadog/Splunk/Elastic* help to consolidate information from various sources and systems together into one consolidated log and console, but these tools only scratch the surface of the complexities involved with distributed logging.



Monolithic Versus Distributed Architectures

Other Distributed Considerations - Distributed transactions

- In monolithic architecture, transaction is straightforward and easy to manage.
- Distributed architectures rely on what is called eventual consistency to ensure the data processed by separate deployment units is at some unspecified point in time all synchronized into a consistent state. This is one of the trade-offs of distributed architecture: high scalability, performance, and availability at the sacrifice of data consistency and data integrity.
- *Transactional sagas* are one way to manage distributed transactions.
  - Sagas utilize either event sourcing for compensation or finite state machines to manage the state of transaction. In addition to sagas, BASE transactions are used. BASE stands for (B)asic availability, (S)oft state, and (E)ventual consistency. BASE transactions are not a piece of software, but rather a technique.
  - Soft state in BASE refers to the transit of data from a source to a target, as well as the inconsistency between data sources. Based on the basic availability of the systems or services involved, the systems will eventually become consistent through the use of architecture patterns and messaging.

Monolithic Versus Distributed Architectures

Other Distributed Considerations - Contract maintenance and versioning

- Another particularly difficult challenge within distributed architecture is contract creation, maintenance, and versioning.
- A contract is behavior and data that is agreed upon by both the client and the service.
- Contract maintenance is particularly difficult in distributed architectures, primarily due to decoupled services and systems owned by different teams and departments.
- Even more complex are the communication models needed for version deprecation.

Chapter 10
Layered Architecture Style

- https://www.oreilly.com/library/view/software-architecture-patterns/9781491971437/ch01.html
- https://magz.techover.io/2023/01/04/kien-truc-phan-tang-layered-architecture-phan-1/
- https://magz.techover.io/2023/01/13/kien-truc-phan-tang-layered-architecture-phan-2/
- https://magz.techover.io/2023/01/19/ap-dung-kien-truc-phan-tang-trong-ung-dung-spring-boot/
- https://www.youtube.com/watch?v=Y9bKZCYxFuI

- Nodejs 3tiers: https://viblo.asia/p/ap-dung-kien-truc-3-layer-architecture-vao-project-nodejs-RQqKLqEmZ7z

Layered Architecture Style

Introduction

- Layered architecture, also known as the n-tiered architecture style, is one of the most common architectural styles. This style of architecture is the de facto standard for most applications, primarily because of its simplicity, familiarity, and low cost.
- It is also a natural way to develop applications due to Conway's law, which states that organizations that design systems are constrained to produce designs that are copies of the communication structures of these organizations.
- Most organizations have user interface (UI) developers, backend developers, rules developers, and database experts (DBAs). These organizational layers fit nicely into the tiers of traditional layered architecture, making it a natural choice for many business applications.
- If a developer or architect is unsure which architecture style they are using, or if an Agile development team "just starts coding," chances are good that it is the layered architecture style they are implementing.

Layered Architecture Style

Topology

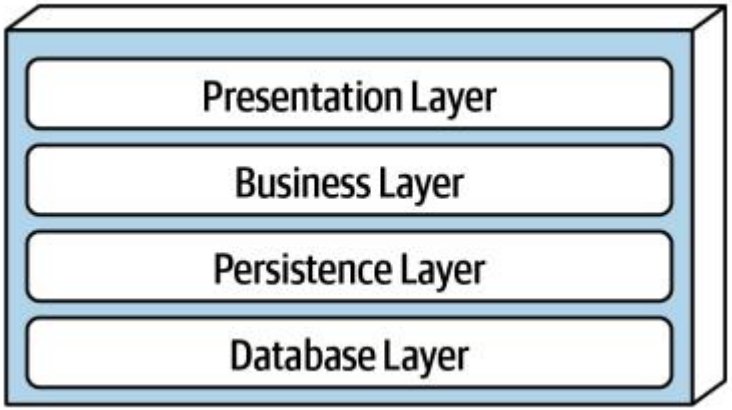


Figure 10-1. Standard logical layers within the layered architecture style

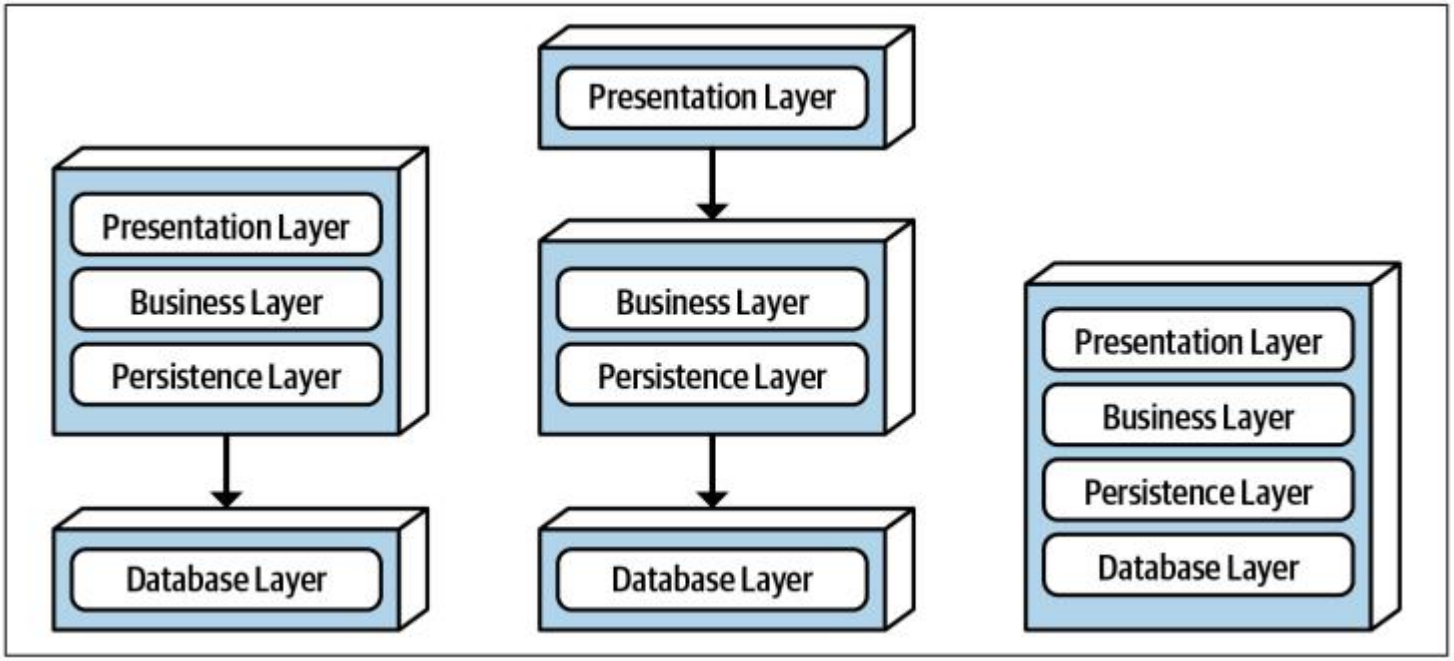


Figure 10-2. Physical topology (deployment) variants

Layered Architecture Style

Layers of Isolation

- Each layer in the layered architecture style can be either *closed* or *open*. A closed layer means that as a request moves top-down from layer to layer, the request cannot skip any layers, but rather must go through the layer immediately below it to get to the next layer.
- The layers of isolation concept means that changes made in one layer of the architecture generally don't impact or affect components in other layers, providing the contracts between those layers remain unchanged.

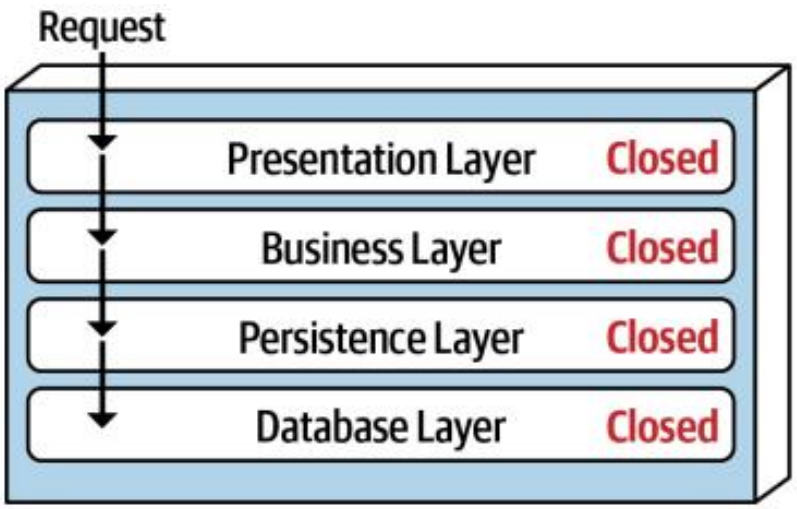


Figure 10-3. Closed layers within the layered architecture

The layers of isolation concept also allows any layer in the architecture to be replaced without impacting any other layer .

Layered Architecture Style

Adding Layers (1/2)

- What happens when a presentation component wants to access shared components inside the business layer?

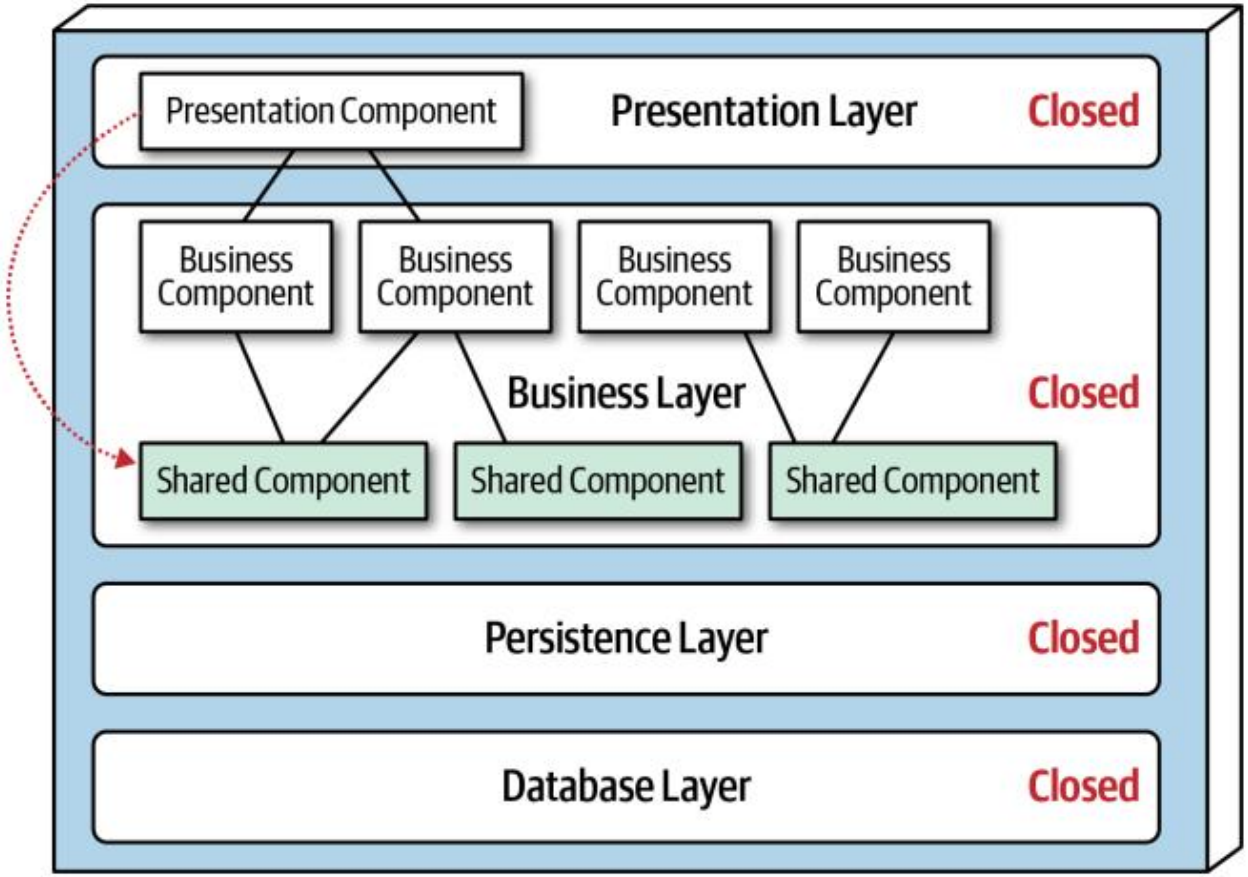
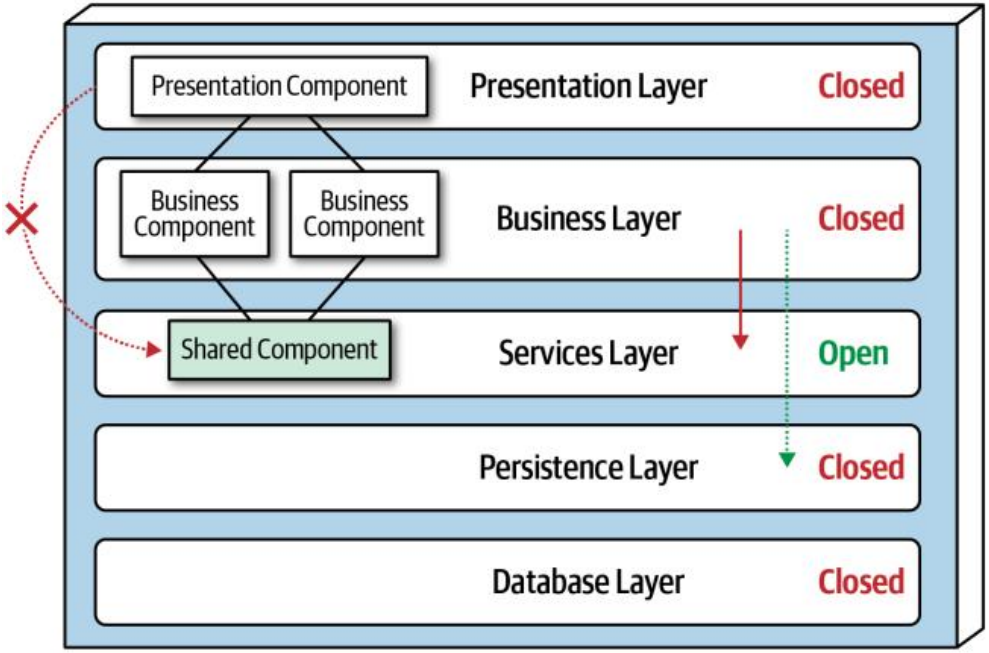


Figure 10-4. Shared objects within the business layer

Layered Architecture Style

Adding Layers (2/2)

- Adding this new layer now architecturally restricts the presentation layer from accessing the shared business objects because the business layer is closed.
- However, the new services layer must be marked as open; otherwise, the business layer would be forced to go through the services layer to access the persistence layer.
- Marking the services layer as open allows the business layer to either access that layer (as indicated by the solid arrow), or bypass the layer and go to the next one down.



Layered Architecture Style

Other Considerations

- Layered architecture makes for a good starting point for most applications when it is not yet known exactly which architectural style will ultimately be used.
- However, when using this technique, keep reuse at a minimum and keep object hierarchies (depth of inheritance tree) fairly shallow to maintain a good level of modularity. This will help facilitate the move to another architectural style later on.
- One thing to watch out for with the layered architecture is the architecture *sinkhole anti-pattern*.
  - The *80-20 rule* is usually a good practice to follow to this situation. It is acceptable if only 20 percent of the requests are sinkholes. However, if 80 percent of the requests are sinkholes, it a good indicator that the layered architecture is not the correct architecture style for the problem domain.
  - Another approach to solving the architecture sinkhole anti-pattern is *to make all the layers in the architecture open*, realizing, of course, that the trade-off is increased difficulty in managing change within the architecture.

Layered Architecture Style

Why Use This Architecture Style

- The layered architecture style is a good choice for small, simple applications or websites.
  - It is also a good architecture choice, particularly as a starting point, for situations with very tight budget and time constraints. Because of the simplicity and familiarity among developers and architects, the layered architecture is perhaps one of the lowest-cost architecture styles, promoting ease of development for smaller applications.
- The layered architecture style is also a good choice when an architect is still analyzing business needs and requirements and is unsure which architecture style would be best.
- As applications using the layered architecture style grow, characteristics like maintainability, agility, testability, and deployability are adversely affected. For this reason, large applications and systems using the layered architecture might be better suited for other, more modular architecture styles.

Layered Architecture Style

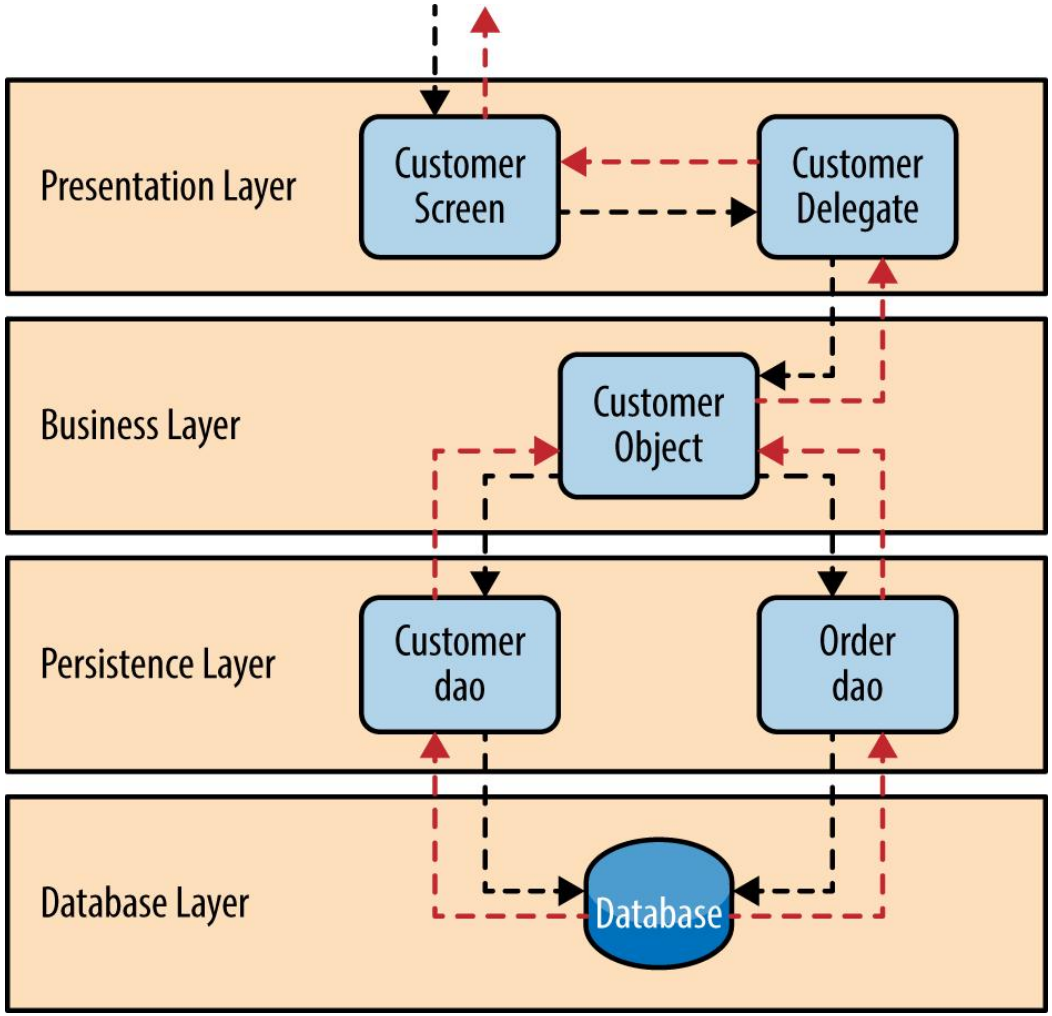
Architecture Characteristics Ratings

Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1
Deployability	★
Elasticity	★
Evolutionary	★
Fault tolerance	★
Modularity	★
Overall cost	★★★★★
Performance	★★
Reliability	★★★
Scalability	★
Simplicity	★★★★★
Testability	★★

- Advantages
  - The framework is simple and easy to learn and implement.
  - There is reduced dependency because the function of each layer is separate from the other layers.
  - Testing is easier because of the separated components; each component can be tested individually.
  - Cost overheads are fairly low.
- Disadvantages
  - Scalability is difficult because the structure of the framework does not allow for growth.
  - They can be difficult to maintain. A change in a single layer can affect the entire system because it operates as a single unit.
  - There is interdependence between layers since a layer depends on the layer above it to receive data.
  - Parallel processing is not possible.

Layered Architecture Style

Pattern Example



Source: O'Reilly

Layered Architecture Style

Summary questions

- 1. What is the difference between an open layer and a closed layer?
- 2. Describe the layers of isolation concept and what the benefits are of this concept.
- 3. What is the architecture sinkhole anti-pattern?
- 4. What are some of the main architecture characteristics that would drive you to use a layered architecture?
- 5. Why isn’t testability well supported in the layered architecture style?
- 6. Why isn’t agility well supported in the layered architecture style?

Chapter 11
Pipeline Architecture Style

Pipeline Architecture Style

Introduction

- One of the fundamental styles in software architecture that repeatedly appears is the pipeline architecture (also known as the pipes and filters architecture).
- This pattern followed as soon as developers and architects decided to split functionality into discrete parts.
- Most developers know this architecture as the underlying principle behind Unix terminal shell languages, such as Bash and Zsh.
- Developers in many functional programming languages will see parallels between language constructs and elements of this architecture. Many tools that utilize the MapReduce programming model follow this basic topology. While these examples show a low-level implementation of the pipeline architecture style, it can also be used for higher-level business applications.

Pipeline Architecture Style

Topology (1)

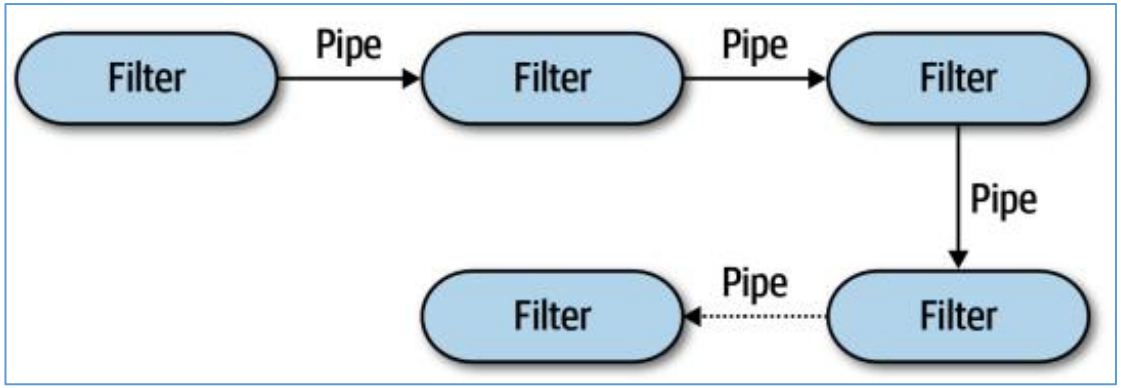
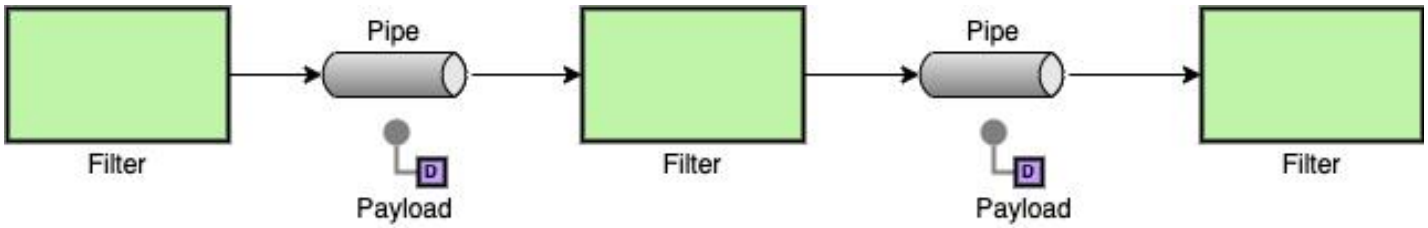


Figure 11-1. Basic topology for pipeline architecture

- Pipes
  - Pipes in this architecture form the communication channel between filters. Each pipe is typically unidirectional and point-to-point (rather than broadcast) for performance reasons, accepting input from one source and always directing output to another. The payload carried on the pipes may be any data format, but architects favor smaller amounts of data to enable high performance.



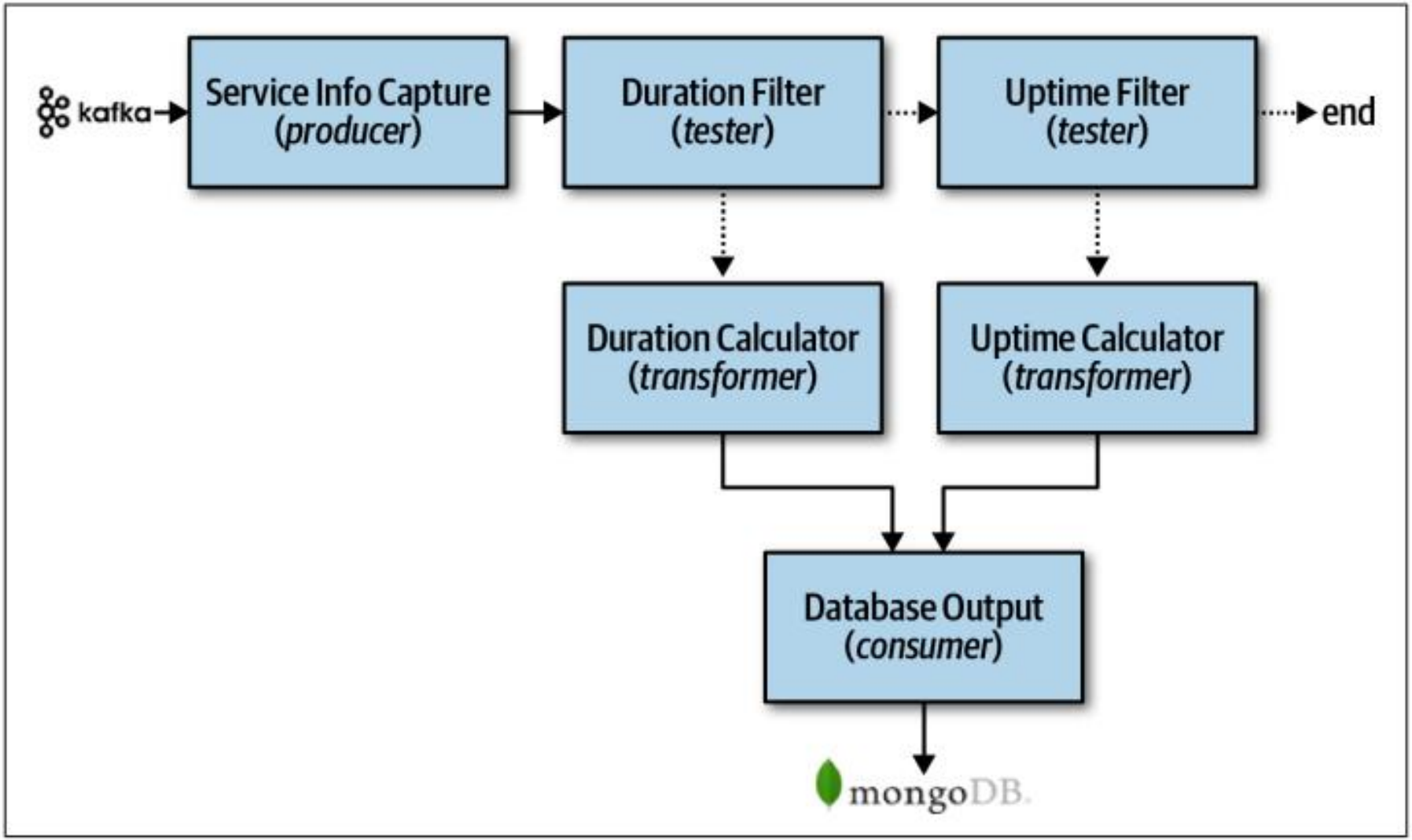
Pipeline Architecture Style

Topology (2) - Filters

- Filters are self-contained, independent from other filters, and generally stateless. Filters should perform one task only. Composite tasks should be handled by a sequence of filters rather than a single one. Four types of filters exist within this architecture style:
- Producer
  - The starting point of a process, outbound only, sometimes called the source.
- Transformer
  - Accepts input, optionally performs a transformation on some or all of the data, then forwards it to the outbound pipe. Functional advocates will recognize this feature as map.
- Tester
  - Accepts input, tests one or more criteria, then optionally produces output, based on the test. Functional programmers will recognize this as similar to reduce.
- Consumer
  - The termination point for the pipeline flow. Consumers sometimes persist the final result of the pipeline process to a database, or they may display the final results on a user interface screen

Pipeline Architecture Style

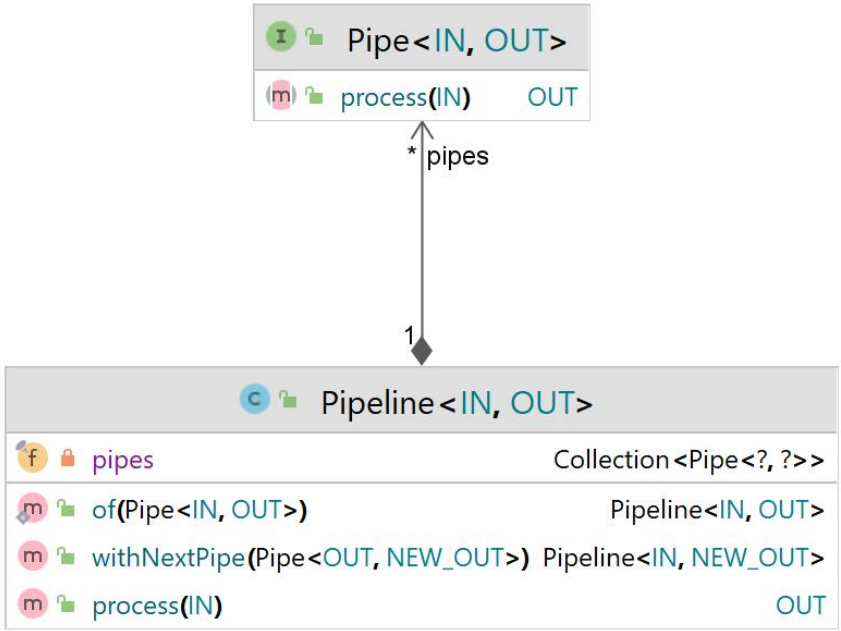
Example



Pipeline Architecture Style

Example 1

```
public interface Pipe<IN, OUT> {
    OUT process(IN input);
}
```




```
public static void main(String[] args) {
    Pipeline<Integer, Long> pipeline =
            Pipeline.of(input ->
input.longValue() * 2);
    pipeline = pipeline
            .withNextPipe(input -> input + 4)
            .withNextPipe(input -> input + 2)
            .withNextPipe(input -> input + 5);
    Long result = pipeline.process(5);
    System.out.println(result);
}
```

Pipeline Architecture Style

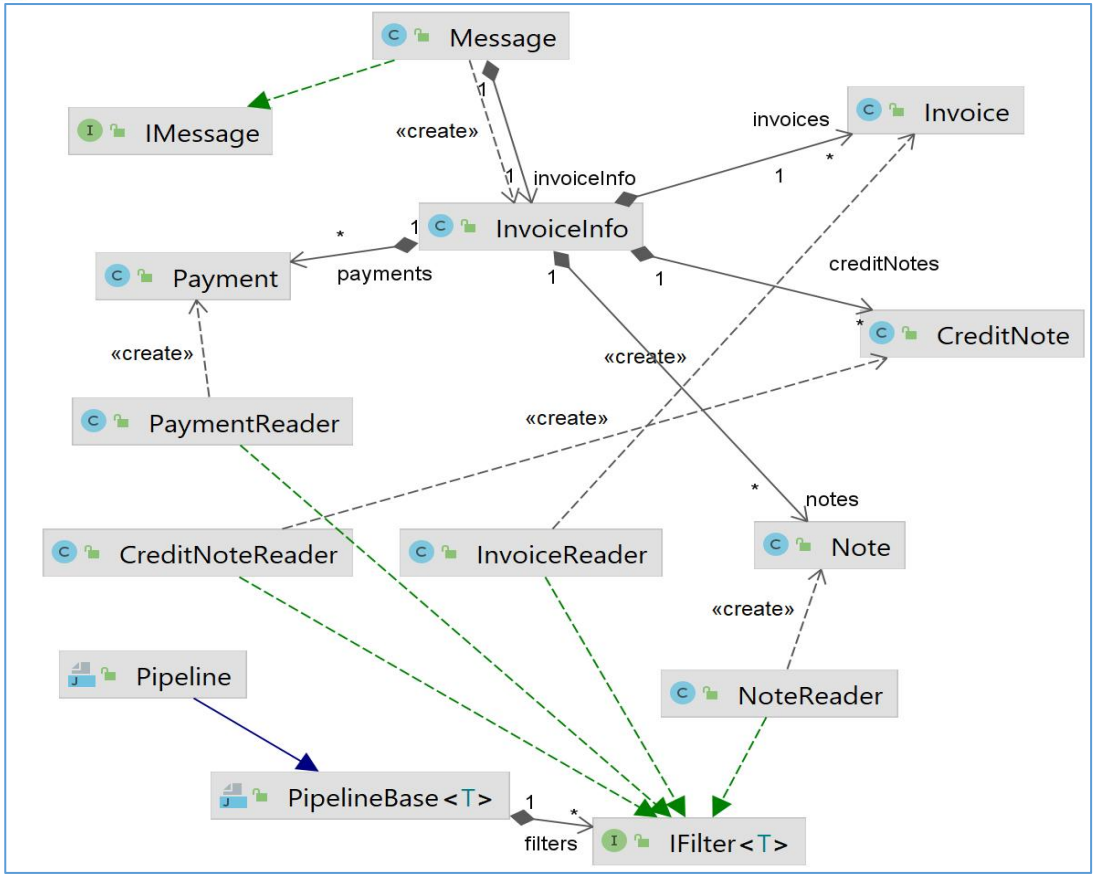
Example 2 – using Function library

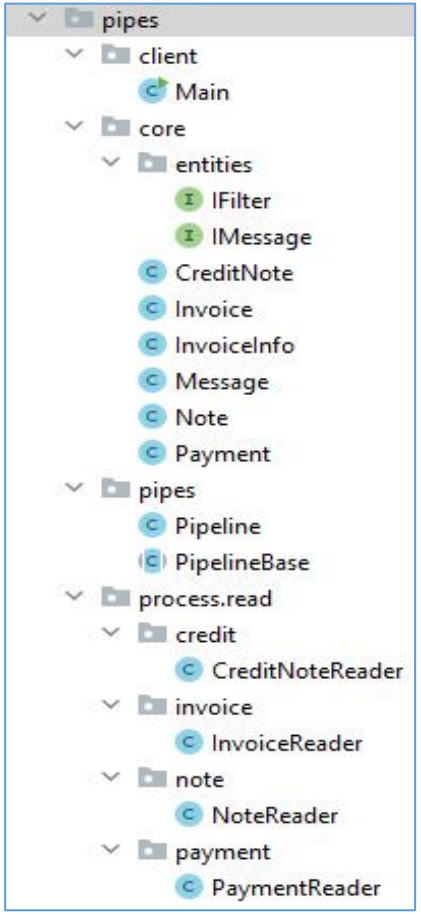
```
public interface Pipe<IN, OUT> {
    OUT process(IN input);

    default <NEW_OUT> Pipe<IN, NEW_OUT> add(Pipe<OUT, NEW_OUT> pipe) {
        return input -> pipe.process(process(input));
    }
}
```

```
import java.util.function.Function;

public class Main {
    public static void main(String[] args) {
        Function<Integer, Integer> square = s -> s * s;
        Function<Integer, Integer> half = s -> s / 2;
        Function<Integer, String> toString =
Object::toString;
        Function<Integer, String> pipeline =
square.andThen(half)
                .andThen(toString);
        String result = pipeline.apply(5);
        System.out.println(result);
    }
}
```

Pipeline Architecture Style

Example 3





Pipeline Architecture Style

Architecture Characteristics Ratings

Architecture characteristic	Star rating
Partitioning type	Technical
Number of quanta	1
Deployability	★★
Elasticity	★
Evolutionary	★★★
Fault tolerance	★
Modularity	★★★
Overall cost	★★★★★
Performance	★★
Reliability	★★★
Scalability	★
Simplicity	★★★★★
Testability	★★★

Chapter 12
Microkernel Architecture Style

Microkernel Architecture Style

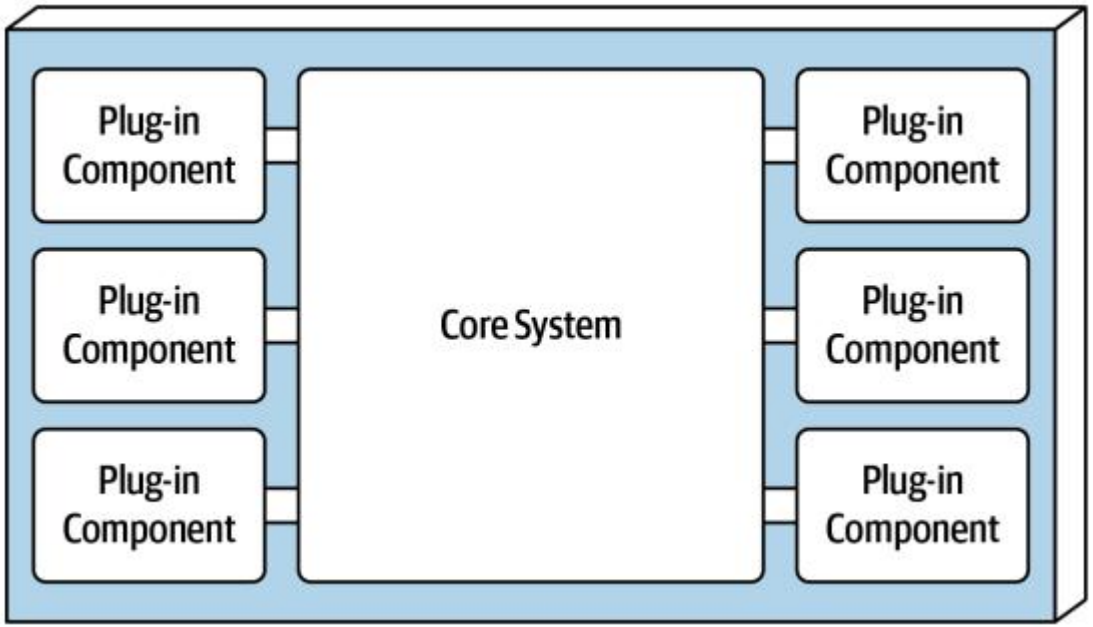
Introduction

- The microkernel architecture style (also referred to as the plug-in architecture) was coined several decades ago and is still widely used today.
- This architecture style is a natural fit for product-based applications (packaged and made available for download and installation as a single, monolithic deployment, typically installed on the customer's site as a third-party product) but is widely used in many nonproduct custom business applications as well.

Microkernel Architecture Style

Topology

- The microkernel architecture style is a relatively simple monolithic architecture consisting of two architecture components: a core system and plug-in components.
- Application logic is divided between independent plug-in components and the basic core system, providing extensibility, adaptability, and isolation of application features and custom processing logic.



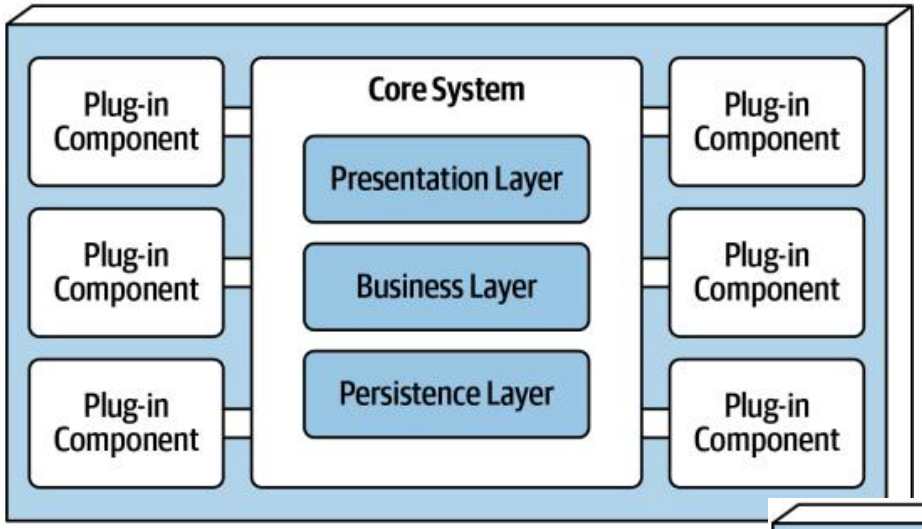
Microkernel Architecture Style

Core System

- The core system is formally defined as the minimal functionality required to run the system.
- Example: Eclipse IDE
  - The core system of Eclipse is just a basic text editor: open a file, change some text, and save the file. It's not until you add plug-ins that Eclipse starts becoming a usable product.
- Core system may be a happy path (general processing flow) through the application, with little or no custom processing.
  - Removing the cyclomatic complexity of the core system and placing it into separate plug-in components allows for better extensibility and maintainability, as well as increased testability.

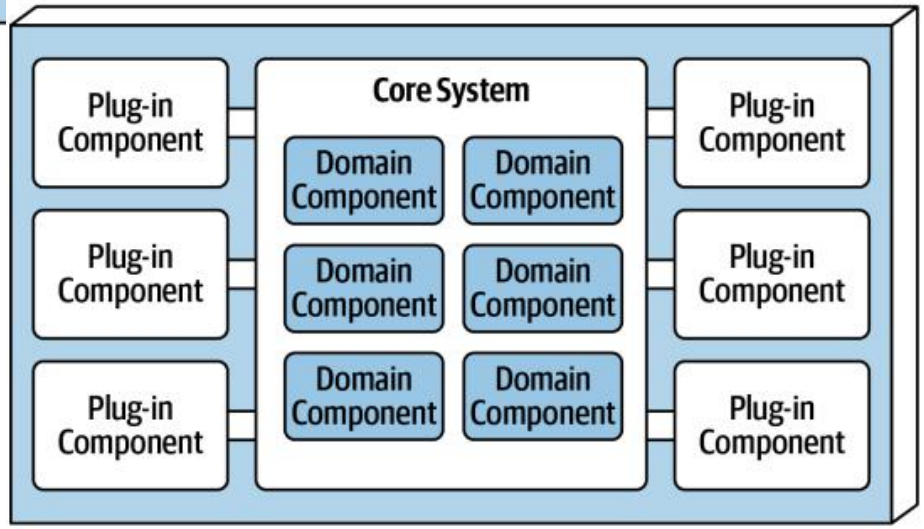
Microkernel Architecture Style

Core - Variations



Layered Core System (Technically Portioned)

Figure 12-2. Variations of the microkernel architecture core system

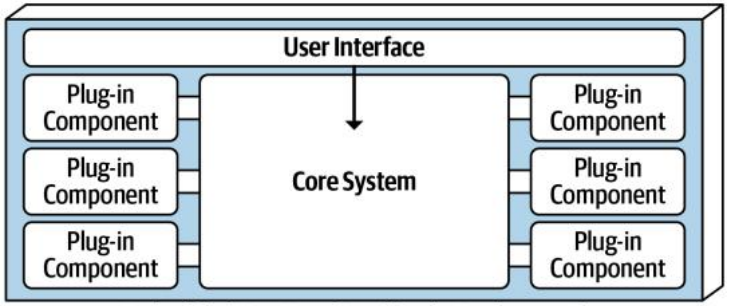


Modular Core System (Domain Portioned)

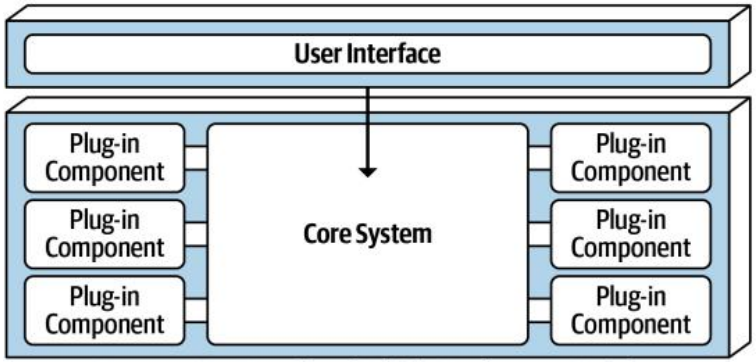
Microkernel Architecture Style

Core System - Presentation layer

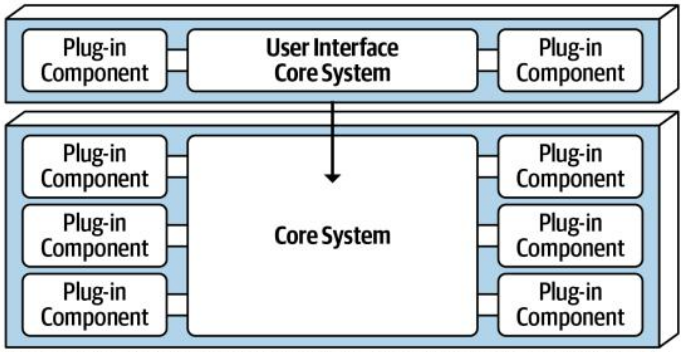
- The presentation layer of the core system can be embedded within the core system or implemented as a separate user interface, with the core system providing backend services.
- As a matter of fact, a separate user interface can also be implemented as a microkernel architecture style. Figure 12-3 illustrates these presentation layer variants in relation to the core system



Embedded User Interface (Single Deployment)



Separate User Interface (Multiple Deployment Units)



Separate User Interface (Multiple Deployment Units, Both Microkernel)

Microkernel Architecture Style

Plug-In Components

- Plug-in components are standalone, independent components that contain specialized processing, additional features, and custom code meant to enhance or extend the core system.
- Additionally, they can be used to isolate highly volatile code, creating better maintainability and testability within the application. Ideally, plug-in components should be independent of each other and have no dependencies between them.
- The communication between the plug-in components and the core system is generally point-to-point, meaning the "pipe" that connects the plug-in to the core system is usually a method invocation or function call to the entry-point class of the plug-in component.
- In addition, the plug-in component can be either compile-based or runtime-based.
  - Runtime plug-in components can be added or removed at runtime without having to redeploy the core system or other plug-ins, and they are usually managed through frameworks such as Open Service Gateway Initiative (OSGi) for Java, Penrose (Java), Jigsaw (Java), or Prism (.NET).
  - Compile-based plug-in components are much simpler to manage but require the entire monolithic application to be redeployed when modified, added, or removed

Microkernel Architecture Style

Plug-In Components – P2P

- Point-to-point plug-in components can be implemented as shared libraries (such as a JAR, DLL, or Gem), package names in Java, or namespaces in C#. Continuing with the electronics recycling assessment application example, each electronic device plugin can be written and implemented as a JAR, DLL, or Ruby Gem (or any other shared library)

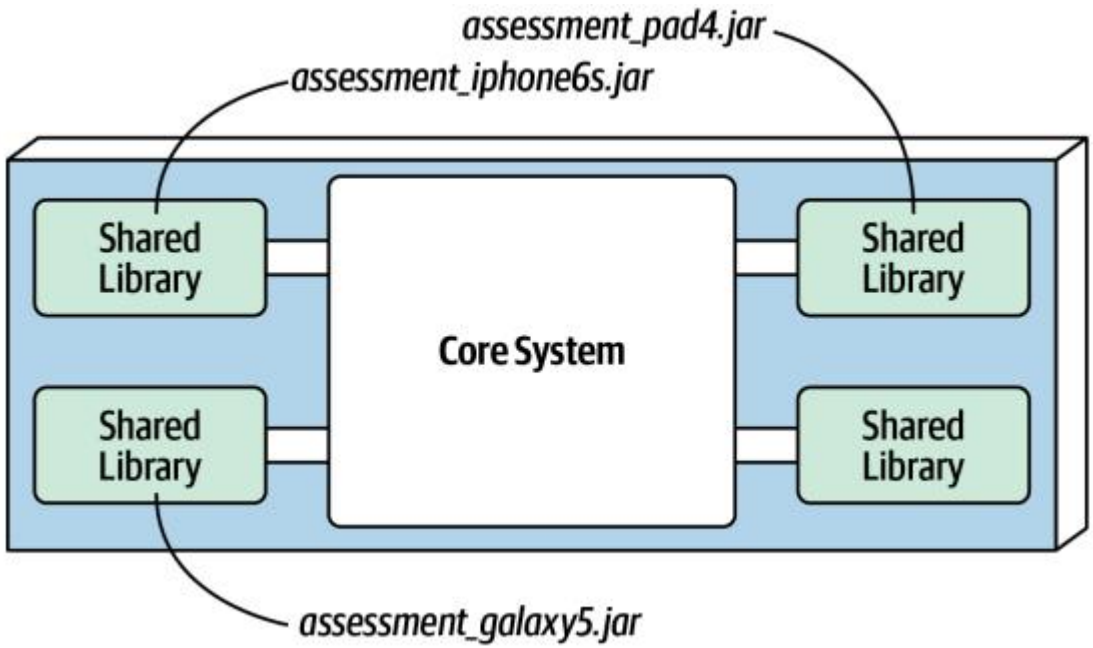


Figure 12-5. Package or namespace plug-in implementation

Microkernel Architecture Style

Plug-In Components – remote

- This type of plugin may including using REST or messaging to invoke plug-in functionality, with each plug-in being a standalone service (or maybe even a microservice implemented using a container).
- Although this may sound like a good way to increase overall scalability, note that this topology is still only a single architecture quantum due to the monolithic core system. Every request must first go through the core system to get to the plug-in service.

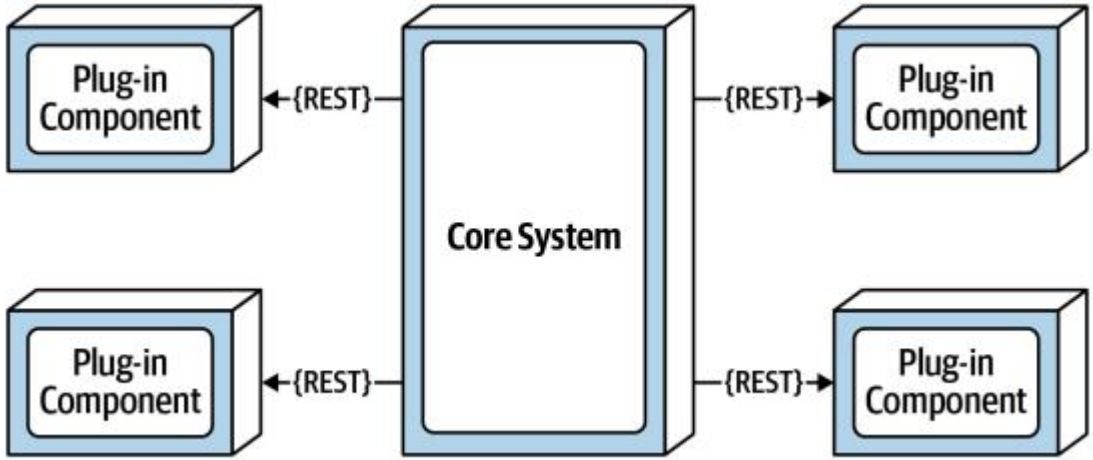


Figure 12-6. Remote plug-in access using REST

Microkernel Architecture Style

Plug-In Components

- With these benefits comes trade-offs. Remote plug-in access turns the microkernel architecture into a distributed architecture rather than a monolithic one, making it difficult to implement and deploy for most third-party on-prem products.

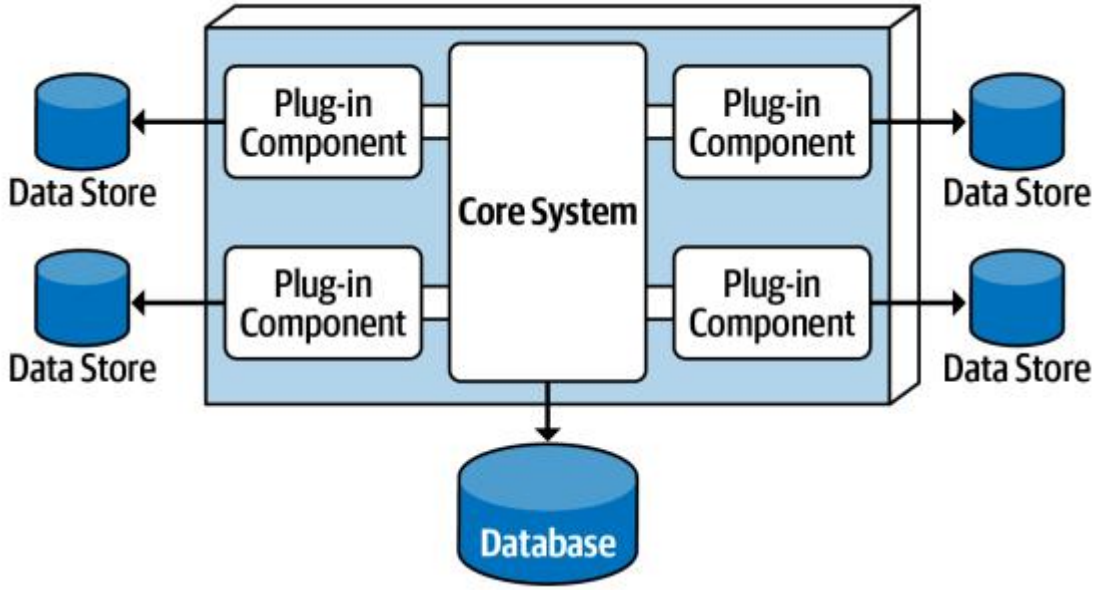


Figure 12-7. Plug-in components can own their own data store

Microkernel Architecture Style

Registry

- The core system needs to know about which plug-in modules are available and how to get to them.
  - One common way of implementing this is through a plug-in registry.
  - This registry contains information about each plug-in module, including things like its name, data contract, and remote access protocol details (depending on how the plug-in is connected to the core system).
- The registry can be as simple as an internal map structure owned by the core system containing a key and the plug-in component reference, or it can be as complex as a registry and discovery tool either embedded within the core system or deployed externally (such as Apache ZooKeeper or Consul).

```
Map<String, String> registry = new HashMap<String, String>();
```

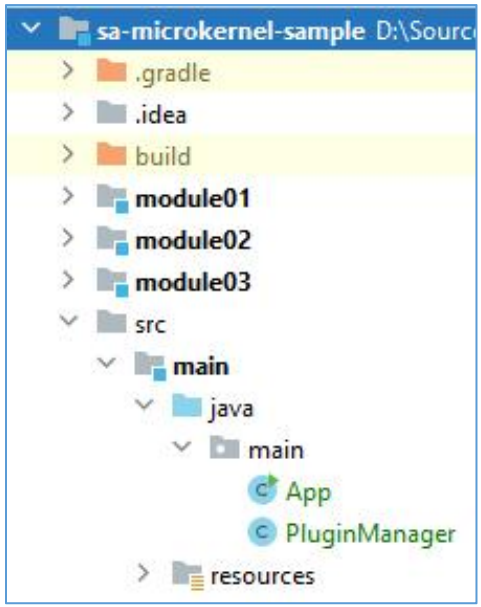
Microkernel Architecture Style

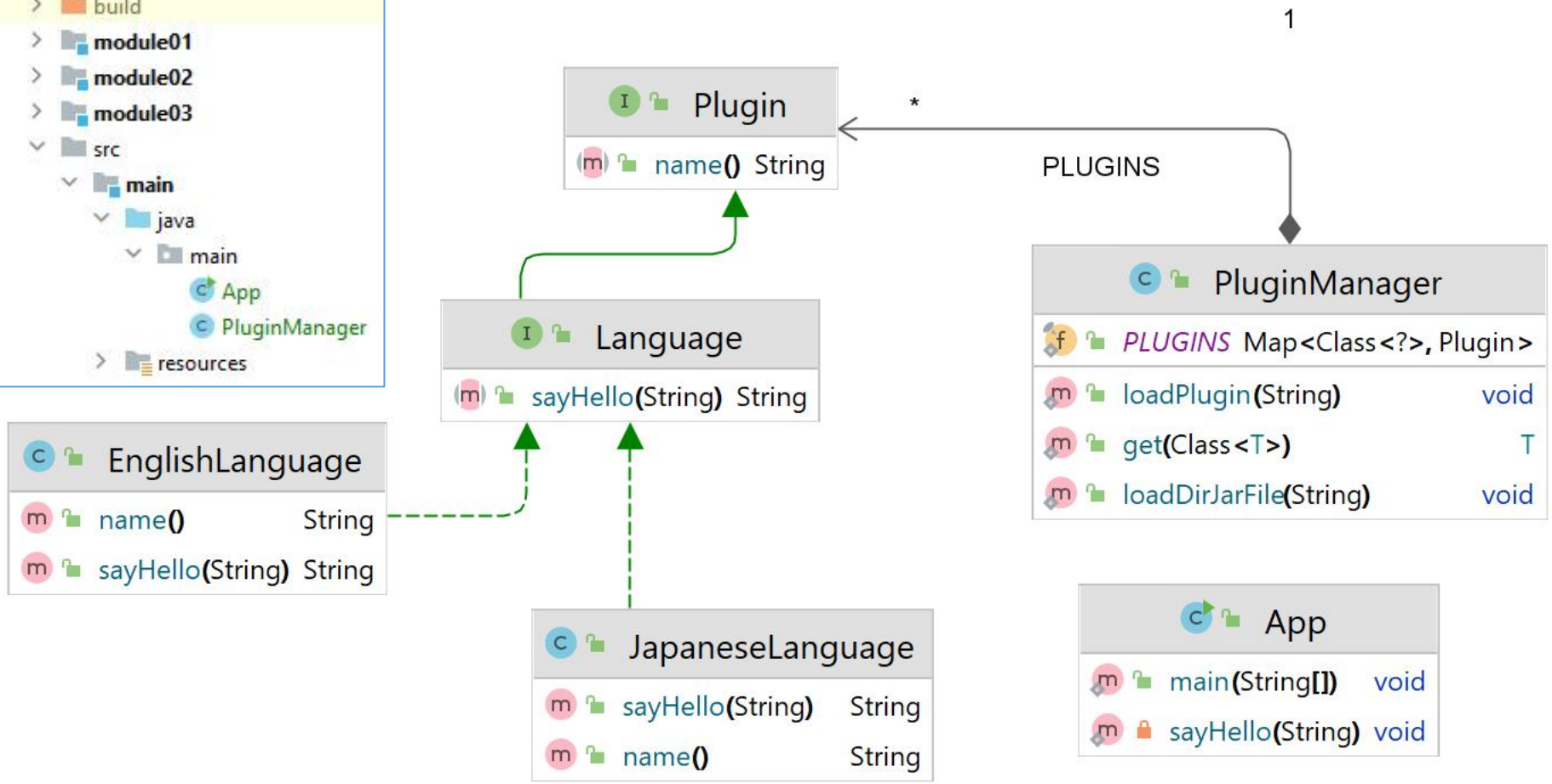
Rating

Architecture characteristic	Star rating
Partitioning type	Domain and technical
Number of quanta	1
Deployability	★★★
Elasticity	★
Evolutionary	★★★
Fault tolerance	★
Modularity	★★★
Overall cost	★★★★★
Performance	★★★
Reliability	★★★
Scalability	★
Simplicity	★★★★
Testability	★★★

Microkernel Architecture Style

Example





Thank you
for your
cooperation!